

Date:10-Aug-26

NumPy & Pandas Practice Task Sheet

Part A: Core Coding Questions (1 to 5)

Q.1 The NumPy Array & Matrix Logic Drill

Focus: Indexing, slicing, boolean masking, and vectorized operations without loops.

- **Task 1:** Create a 5×5 matrix with random integers between 1 and 50 using `np.random.randint`.
- **Task 2:** Find and print all numbers in the matrix that are greater than 25 (using boolean indexing).
- **Task 3:** Replace all even numbers in that matrix with 0 without altering the original matrix layout structure.
- **Task 4:** Normalize a 1D random array of 10 elements so that all its values fall strictly between 0 and 1 using the formula: $\frac{x - \min(x)}{\max(x) - \min(x)}$.

Q.2 The Pandas Data Cleaning & Transformation Challenge

Focus: DataFrames, handling missing values, filtering, and aggregations.

- **Task 1:** Create a Pandas DataFrame containing columns Employee_ID, Name, Department, Age, and Salary with at least 6 rows of sample data, intentionally including a few NaN values in the Salary column.
- **Task 2:** Detect and count the total number of missing values in each column using `.isnull().sum()`.
- **Task 3:** Fill the missing Salary values with the median salary of their respective Department using `groupby` and `fillna`.
- **Task 4:** Create a new calculated column called Annual_Bonus which is 10% of their Salary, and group the data by Department to find the average salary per department.

Q.3 NumPy Array Reshaping & Slicing Challenge

Focus: Reshaping dimensions and extracting specific sub-grids.

- **Task 1:** Create a 1D NumPy array containing numbers from 1 to 20 (inclusive).
- **Task 2:** Reshape this 1D array into a 4×5 matrix.
- **Task 3:** From this matrix, write code to extract and print only the **2nd and 3rd columns**.
- **Task 4:** Extract the four corner elements of the matrix into a single 1D array.

Q.4 Pandas Filtering & Conditional Selection

Focus: Complex boolean indexing and conditional updates in DataFrames.

- **Task 1:** Create a Pandas DataFrame with columns Product, Category, Price, and Stock with at least 5 rows.
- **Task 2:** Write code to filter and display only the rows where the Price is greater than 500 and the Stock is less than 50.
- **Task 3:** Add a new boolean column named Restock_Needed which evaluates to True if Stock is less than 20, and False otherwise.
- **Task 4:** Sort the final DataFrame in descending order based on Price.

Q.5 Vectorized Temperature & Math Operations

Focus: High-performance vectorization without Python for loops.

- **Task 1:** Generate an array of 10 random floating-point temperatures in Celsius using `np.random.uniform` (ranging from 15.0 to 40.0).
- **Task 2:** Convert all of them to Fahrenheit simultaneously using vectorization with the formula: $F = \left(C \times \frac{9}{5}\right) + 32$.
- **Task 3:** Calculate the square root of all values in a separate 1D array of integers from 1 to 10 using `np.sqrt()`.
- **Task 4:** Find the index position of the maximum temperature in your Fahrenheit array using `np.argmax()`.

Part B: Debugging Challenges (6 to 7)

Q.6 Debugging Question 1 (NumPy Boolean Masking)

Focus: Fixing logical operator errors in vectorized environments.

- **The Problem:** A student wants to filter a NumPy array to keep only the numbers between 20 and 50, but their code is throwing an error. Find the bug, explain why it failed, and provide the corrected code.

Python

```
import numpy as np

arr = np.array([10, 25, 30, 45, 60, 15, 50])

filtered_arr = arr[(arr > 20) and (arr < 50)]

print(filtered_arr)
```

- **Hint to provide them:** Python's logical and keyword evaluates truth values of entire objects rather than element-by-element. Look into using bitwise operators (`&`) for element-wise masking in NumPy.

Q.7 Debugging Question 2 (Pandas Column Reference)

Focus: Resolving KeyError exceptions due to mismatched column labels.

- **The Problem:** A student is trying to drop a column from a DataFrame, but Python raises a `KeyError`. Find the bug, explain why it failed, and provide the corrected code.

Python

```
import pandas as pd
```

```
data = {'Employee': ['Ravi', 'Priya', 'Amit'], 'Department': ['IT', 'HR', 'Finance'], 'Salary': [50000, 60000, 55000]}
```

```
df = pd.DataFrame(data)
```

```
df_dropped = df.drop('Salaries', axis=1)
```

```
print(df_dropped)
```

- **Hint to provide them:** Always check the exact string spelling and column names defined in your dictionary source (`df.columns`).